

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Отчёт
По лабораторной работе №4
по дисциплине Компьютерная графика
Вариант: -

Работу выполнили:
Молчанов Федор Денисович Р3313
Метель Леонард Валерьевич Р3312
Кравченко Дмитрий Олегович Р3319
Работу принял:
Жданов Дмитрий Дмитриевич

г. Санкт-Петербург 2026

Лабораторная работа N 4

Формирование изображения методом трассировки путей

1. Цель работы

Цель лабораторной работы состоит в освоении метода синтеза изображений трехмерных сцен с учетом глобального освещения. Согласно заданию из файла ЛР3_4.docx, требуется построить изображение сцены, заданной треугольной сеткой, обеспечить физически корректную модель отражения, использовать протяженные источники света, антиалиасинг, нормировку яркости, гамма-коррекцию и сохранить результат в формате PPM.

Практическая реализация выполнена в файле `lr4_fixed.py`. Файл `lr4_path_tracer.py` является исходной версией с TODO, а в `lr4_fixed.py` исправлены эти пункты: учет попадания в источник после диффузного отражения, добавление режима Cook-Torrance, параллельный расчет строк и экспорт AOV-буферов для дальнейшей фильтрации в LР5.

2. Математическая модель

Основой трассировки путей является уравнение рендеринга:

$$L_{o(x, \omega_o)} = L_e(x, \omega_o) + \int_{\Omega} f_r(x, \omega_i, \omega_o) L_i(x, \omega_i) \max(0, n \cdot \omega_i) d\omega_i$$

Здесь L_o – исходящая яркость, L_e – собственное излучение поверхности, f_r – BRDF материала, L_i – входящая яркость, n – нормаль к поверхности, ω_i и ω_o – входящее и выходящее направления. Интеграл по полусфере оценивается методом Монте-Карло: для каждого пикселя выпускается несколько случайных лучей, после чего оценки усредняются.

Для одного пикселя оценка имеет вид:

$$\hat{L}(p) = \frac{1}{S} \sum_{s=1}^S L_{s(p)}$$

где S – число сэмплов на пиксель, а L_s – вклад одного случайного пути. Антиалиасинг достигается случайным смещением координат внутри пикселя:

$$x_s = \frac{x + \xi_1}{W}, \quad y_s = \frac{y + \xi_2}{H}, \quad \xi_1, \xi_2 \in [0, 1)$$

3. Геометрия и пересечение лучей

Сцена описана набором треугольников. В `lr4_fixed.py` для пересечения луча с треугольниками используется векторизованный вариант алгоритма Moller-Trumbore. Луч задается параметрически:

$$r(t) = o + td, \quad t > \varepsilon$$

где o – начало луча, d – нормализованное направление. Для каждого треугольника с вершинами v_0, v_1, v_2 проверяется существование барицентрических координат:

$$p = (1 - u - v)v_0 + uv_1 + vv_2, \quad u \geq 0, \quad v \geq 0, \quad u + v \leq 1$$

Из всех корректных пересечений выбирается ближайшее значение t . Малое смещение ε используется при порождении вторичных и теневых лучей, чтобы избежать самопересечения поверхности.

4. Модель материалов

В задании указано, что поверхность может одновременно иметь диффузную и зеркальную составляющие, а выбор события должен выполняться методами важностной выборки и русской рулетки. В программе материал задается параметрами `diffuse`, `mirror`, `roughness`, `metallic` и `emission`.

Для диффузного отражения применяется модель Ламберта:

$$f_d = \frac{k_d}{\pi}$$

Новое направление выбирается по косинусному распределению над полусферой:

$$p(\omega_i) = \frac{\max(0, n \cdot \omega_i)}{\pi}$$

Такой выбор уменьшает дисперсию оценки, потому что PDF пропорциональна косинусному множителю из уравнения рендеринга.

В исправленной версии добавлен режим `cook_torrance`. В нем зеркальная часть описывается микрофакетной моделью:

$$f_{s(\omega_i, \omega_o)} = \frac{D(h)G(\omega_i, \omega_o)F(\omega_o, h)}{4 \max(0, n \cdot \omega_i) \max(0, n \cdot \omega_o)}$$

Распределение микронормалей GGX вычисляется как:

$$D(h) = \frac{\alpha^2}{\pi(\cos^2 \theta_h(\alpha^2 - 1) + 1)^2}, \quad \alpha = \text{roughness}^2$$

Френелевский множитель реализован приближением Шлика:

$$F = F_0 + (1 - F_0)(1 - \cos \theta)^5$$

Итоговая BRDF в режиме `Cook-Torrance` является суммой диффузной и микрофакетной частей. При этом диффузная энергия уменьшается для металлических и сильно зеркальных материалов, что помогает соблюдать физическое ограничение: суммарное отражение по каждому RGB-каналу не должно превышать единицу.

5. Трассировка пути

Функция `trace_path` накапливает яркость вдоль пути. В начале $L = 0$, а текущий вес пути равен $\beta = (1, 1, 1)$. После каждого взаимодействия с поверхностью вклад обновляется по формуле:

$$\beta_{\text{new}} = \beta_{\text{old}} \frac{f_{r(\omega_i, \omega_o)} \max(0, n \cdot \omega_i)}{p(\omega_i) p_{\text{event}}}$$

Здесь $p(\omega_i)$ – плотность выбранного направления, а p_{event} – вероятность выбранного типа события: диффузного, зеркального или поглощения.

Для ограничения длины путей используется русская рулетка. Начиная с заданной глубины путь продолжается с вероятностью:

$$p_{\text{survive}} = \min(0.99, \max(0.05, \max(\beta_r, \beta_g, \beta_b)))$$

Если путь выживает, его вес делится на p_{survive} . Благодаря этому математическое ожидание оценки не изменяется, но уменьшается число малозначимых продолжений.

6. Прямое освещение и MIS

Прямой свет вычисляется методом Next Event Estimation: из точки пересечения выбирается случайная точка на протяженном источнике, после чего проверяется видимость теневым лучом. Вклад источника имеет вид:

$$L_{\text{direct}} = L_e f_r \frac{\cos \theta_x \cos \theta_l}{r^2} \frac{1}{p_{\text{light}} p_{\text{area}}}$$

В исходной версии с TODO корректно учитывались попадания в источник только для луча камеры и после зеркального события. В `lr4_fixed.py` добавлен учет случая, когда предыдущее событие было диффузным. Для этого используется Multiple Importance Sampling:

$$w_{\text{mis}} = \frac{p_{\text{bsdf}}}{p_{\text{bsdf}} + p_{\text{light}}}$$

Если путь после диффузного отражения попадает в источник, его вклад добавляется как:

$$L = L + \beta L_e w_{\text{mis}}$$

Это устраняет двойной учет света между стратегиями BRDF-сэмплинга и явного сэмплинга источника.

7. Реализация рендера

Класс `PathTracer` вычисляет изображение построчно. Для каждого пикселя он выпускает `spp` лучей с независимым генератором случайных чисел. Исправленная версия поддерживает параметр `workers`: если он больше единицы, строки обрабатываются параллельно через `ThreadPoolExecutor`. Поскольку каждая строка записывается в отдельный локальный буфер, параллельная сборка результата не требует общей записи в один пиксель.

Кроме итогового HDR-кадра программа сохраняет вспомогательные AOV-буферы:

- `direct` – прямое освещение первого попадания;
- `secondary` – вторичное и косвенное освещение;
- `depth` – глубина первого пересечения;
- `normal` – нормаль в точке первого пересечения;
- `object_id` – идентификатор объекта.

Эти данные нужны для LР5, где шум удаляется не только по цвету, но и с учетом геометрии сцены.

8. Тональная компрессия и сохранение

Сначала изображение формируется в абсолютных HDR-яркостях. Затем оно переводится в диапазон $[0, 1]$ выбранной нормировкой, обрезается и подвергается гамма-коррекции:

$$V_{\text{corr}} = V^{\frac{1}{\gamma}}, \quad \gamma = 2.2$$

После этого значения масштабируются в диапазон $0..255$ и записываются в PPM:

$$\text{RGB}_8 = \text{round}(255V_{\text{corr}})$$

9. Результаты

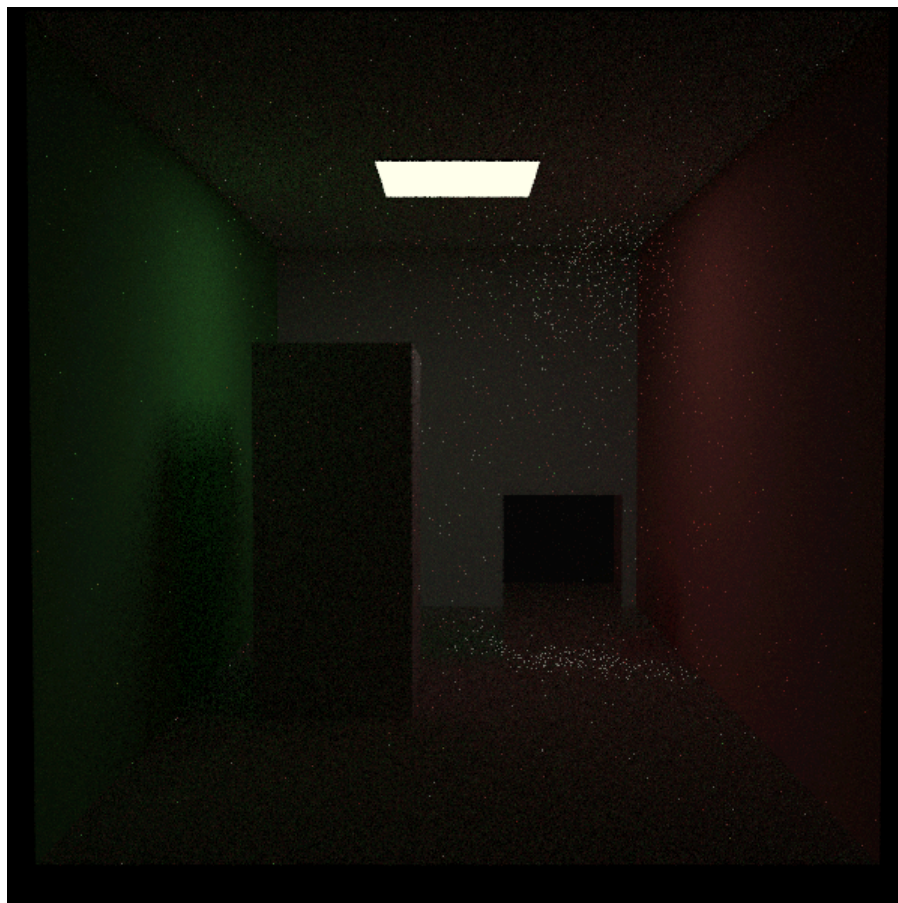


Рис. 1. Результат рендера сцены. Исходный файл результата: renders/render.ppm.

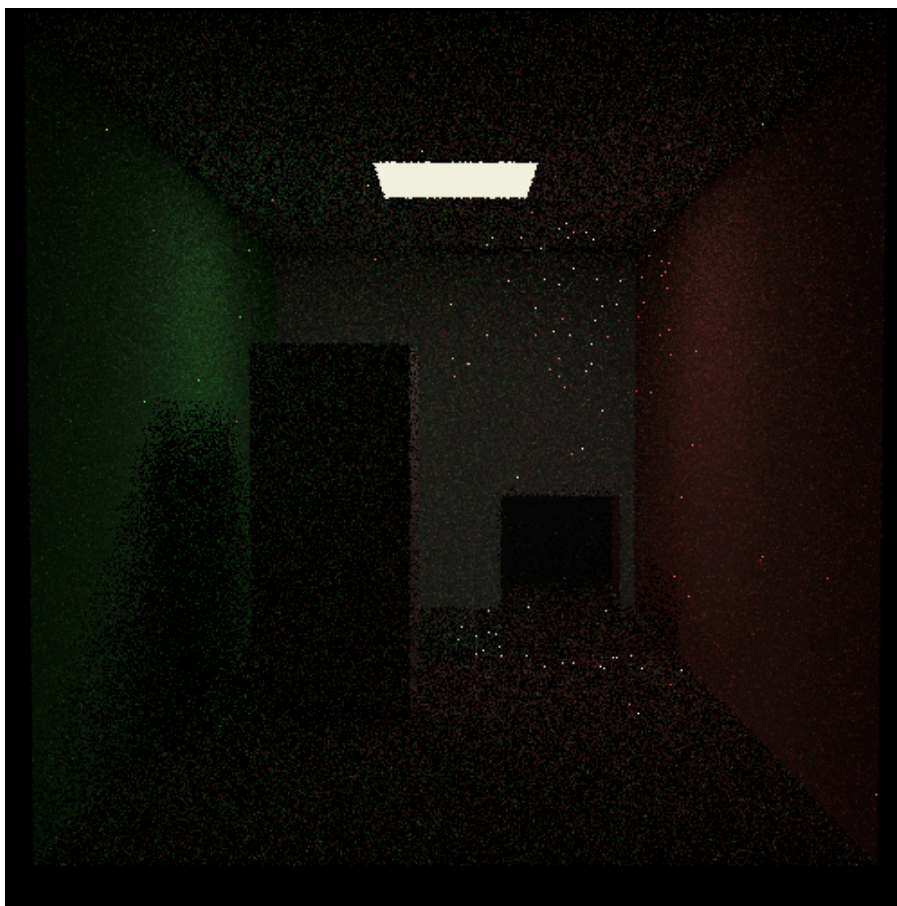


Рис. 2. Рендер разрешением 500x500. Исходный файл результата: renders/
manual_500_spp1.ppm.

Полученные изображения демонстрируют работу глобального освещения: видны мягкие тени от протяженного источника, цветовые переотражения от стен и накопление непрямого освещения. Шум является ожидаемым следствием Монте-Карло оценки и уменьшается при росте числа сэмплов на пиксель.

10. Вывод

В работе реализован трассировщик путей для треугольной сцены с RGB-материалами, диффузным и зеркальным отражением, протяженными источниками света, антиалиасингом и гамма-коррекцией. Исправления в `lr4_fixed.py` делают оценку физически корректнее: добавлен MIS для диффузного попадания в источник, реализован режим Cook-Torrance, поддержана параллельная обработка строк и сохранение AOV-буферов. Результаты сохранены в `.ppm` и могут использоваться как входные данные для последующей фильтрации.